

UD01_Actividades

Configurar un Token de Acceso Personal (PAT) en GitHub

1. Entra en GitHub e inicia sesión
2. Ve a Setting > Developer Settings > Credentials > Personal Access Tokens > Tokens (classic).
3. Genera un nuevo token y asignarle una descripción, selecciona el alcance repo y genera el token.
4. Copia y guarda el token para usarlo como contraseña cuando Git te pida credenciales.

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

UD01_Actividades

What's this token for?

Expiration

📅 30 days (Nov 04, 2026) ▼

The token will expire on the selected date

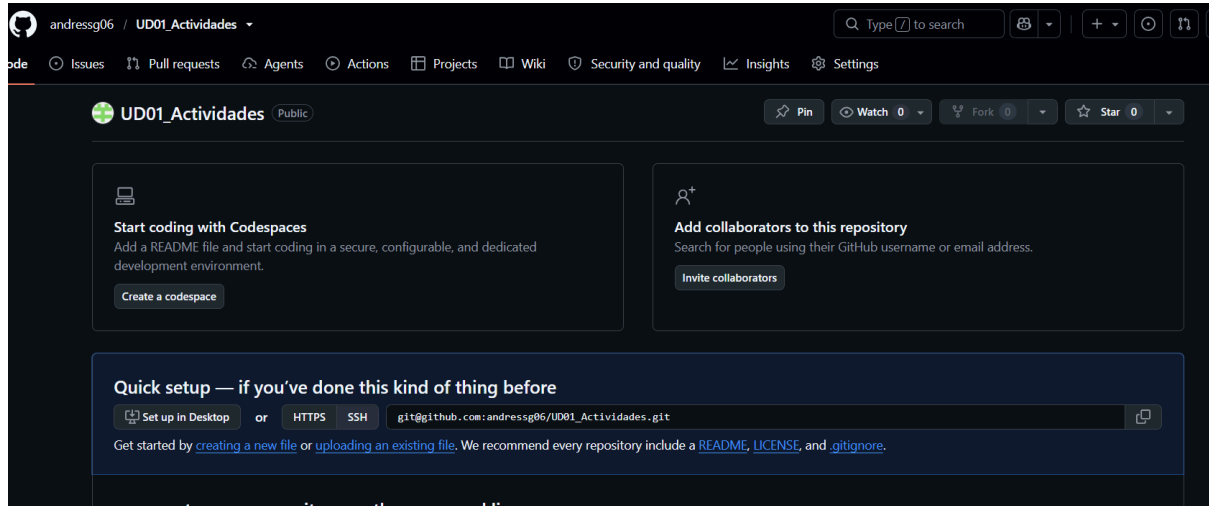
Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events

Crear el repositorio remoto y conectar con local

1. Crea un nuevo repositorio GitHub llamado UD01_Actividades (déjalo vacío; sin README, sin .gitignore ni Licencia).
2. En tu repositorio local renombra la rama master a main manteniendo el historial y comprueba que la rama local se ha renombrado a main correctamente.
3. Asocia el repositorio remoto con tu proyecto local



```
andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (master)
$ git branch
* master

andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (master)
$ git branch -M main

andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (main)
$ git branch
* main
```

```
andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (main)
$ git remote -v
origin https://github.com/andressg06/UD01_Actividades.git (fetch)
origin https://github.com/andressg06/UD01_Actividades.git (push)
```

Subir cambios y verificar en GitHub

1. Sube la rama main al repositorio remoto (Si te solicita credenciales, introduce tu usuario de GitHub y pega el Token como contraseña).

Connect to GitHub ×

GitHub
Sign in

Browser/Device Token

Personal access token

Sign in

Don't have an account? [Sign up](#)

```
andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 429 bytes | 429.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/andressg06/UD01_Actividades.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

2. Entra en GitHub y comprueba que se han subido los archivos a la rama main.

UD01_Actividades Public Pin Watch

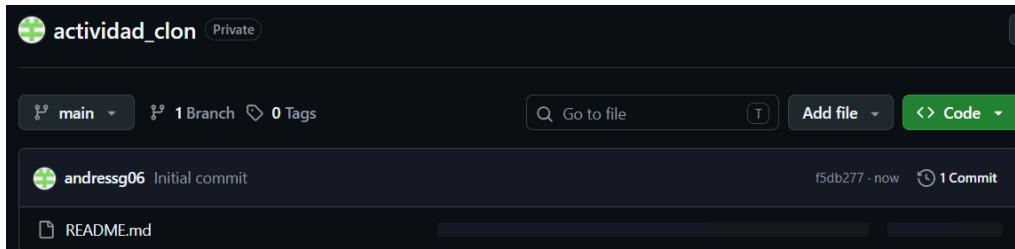
main 1 Branch 0 Tags Add file Code

andressg06 Añadir README y gitignore iniciales 1c55674 · yesterday 1 Commit

.gitignore	Añadir README y gitignore iniciales	yesterday
readme.txt	Añadir README y gitignore iniciales	yesterday

Crear y clonar un segundo repositorio

1. En GitHub, crea un nuevo repositorio llamado actividad_clon y marca la casilla para incluir un README inicial.



2. Copia la URL del repositorio remoto.
3. En tu terminal local (fuera del proyecto anterior), clona el repositorio:

```
andressg_06@andressg06 MINGW64 ~/Desktop
$ git clone https://github.com/andressg06/actividad_clon.git
Cloning into 'actividad_clon'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

4. Comprueba el contenido y la configuración del repositorio:

```
andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ ls
README.md

andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

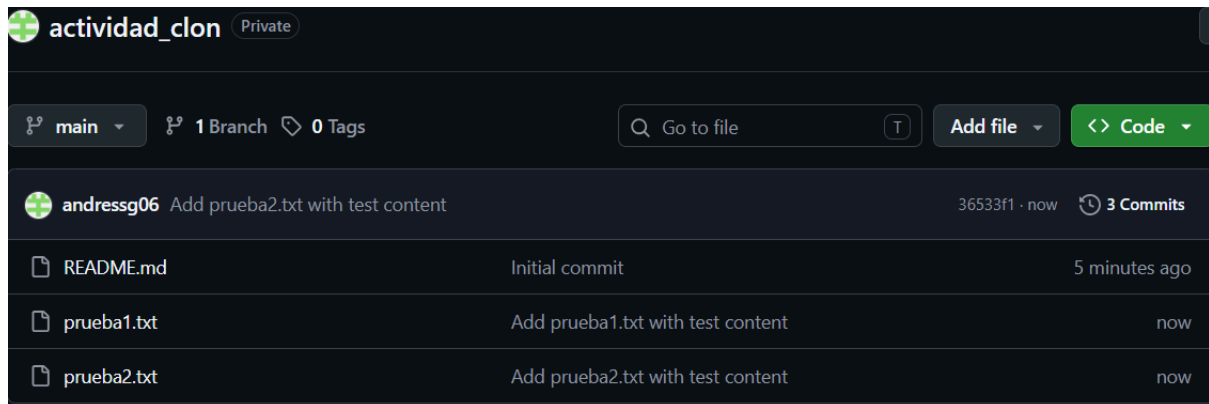
5. Comprueba que estás en la rama main y la URL remota

```
andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ git branch
* main

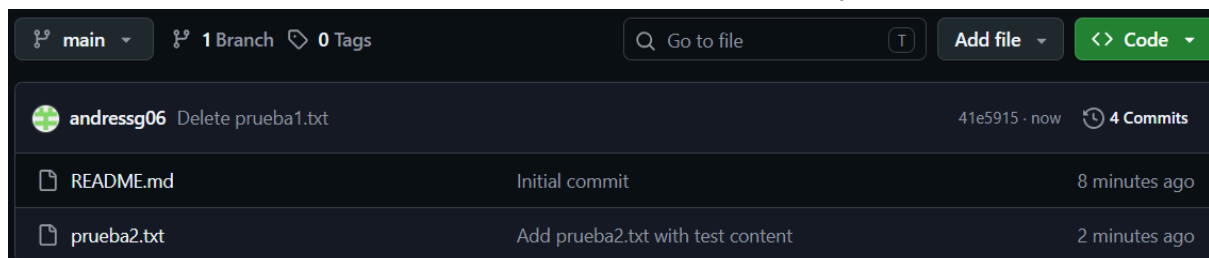
andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ git remote -v
origin  https://github.com/andressg06/actividad_clon.git (fetch)
origin  https://github.com/andressg06/actividad_clon.git (push)
```

Eliminación de un fichero en remoto

1. En el proyecto actividad_clon en GitHub, crea dos archivos prueba1.txt y prueba2.txt y haz commit.



2. A continuación, elimina el archivo prueba1.txt de GitHub y confirma el commit.



Sincronizar los cambios con el repositorio local

1. En tu terminal local, descarga la versión más reciente del repositorio remoto del proyecto actividad_clon
2. Comprueba cómo se ha actualizado la carpeta local y que el archivo prueba1.txt no está.

```
andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ git pull origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (8/8), 2.65 KiB | 246.00 KiB/s, done.
From https://github.com/andressg06/actividad_clon
* branch main -> FETCH_HEAD
f5db277..41e5915 main -> origin/main
Updating f5db277..41e5915
Fast-forward
 prueba2.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 prueba2.txt

andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ ls
README.md prueba2.txt
```

3. Comprueba tu historial local ¿Qué aparece registrado sobre el archivo prueba1.txt?

```
andressg_06@andressg06 MINGW64 ~/Desktop/actividad_clon (main)
$ git log --oneline
41e5915 (HEAD -> main, origin/main, origin/HEAD) Delete prueba1.txt
36533f1 Add prueba2.txt with test content
e6ee2d0 Add prueba1.txt with test content
f5db277 Initial commit
```

Crear una rama de trabajo y colaborar

1. Vuelve al directorio del primer proyecto UD01_Actividades
2. Crea y cámbiate a una nueva rama con el nombre del rama01_XXX (Dónde XXX son tus iniciales)

```
andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (main)
$ git switch -c rama01_AGO
Switched to a new branch 'rama01_AGO'

andressg_06@andressg06 MINGW64 ~/Desktop/Proyecto_PPS (rama01_AGO)
$ git branch
main
* rama01_AGO
```

3. Añade a esta rama el documento con las capturas que has ido haciendo durante la actividad.
4. Ve a Settings > Collaborators > Add people e invita a la profesora pilarpr

Fusionar la rama con la principal

1. Cámbiate a la rama main
2. Fusiona tu rama de trabajo en main
3. Sube los cambios actualizados al repositorio remoto

Generar y resolver conflictos de fusión (Merge Conflicts)

1. Crea y cámbiate a una nueva rama llamada archivo_conflictos
2. Modifica la primera línea del archivo readme.txt en esta rama, guarda los cambios y realiza el commit
3. Vuelve a la rama main
4. Modifica la misma primera línea del archivo readme.txt de forma distinta, guarda y realiza el commit
5. Intenta fusionar la rama archivo_conflictos en main. ¿Qué ocurre?
6. Resolver el conflicto:
 - Abre readme.txt en tu editor de código. Verás marcas como estas:


```
Plaintext
<<<<<<< HEAD
Texto modificado en main
=====
```

Texto modificado en archivo_conflictos

>>>>>> archivo_conflictos

- Borra las marcas (<<<<<<, =====, >>>>>>) y deja únicamente el texto correcto final.
- Guarda el archivo y completa la fusión en Git

Revisar el historial de commits

1. Desde Git revisa el historial de tus repositorios
2. Desde GitHub, revisa los commits de tus repositorios

Revisa la lista cronológica de commits, comprueba cómo se reflejan las ramas creadas, la resolución del conflicto y las aportaciones de los colaboradores.

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